

Flutter + Android Keystore + CSR + NestJS mTLS Architecture

Overview

This architecture provides enterprise-grade device authentication using:

- Flutter applications
- Android Keystore
- CSR (Certificate Signing Request)
- Private CA
- Mutual TLS (mTLS)
- NestJS backend

The private key never leaves the device.

Architecture Flow

Device generates keypair INSIDE Android Keystore

↓

Private key NEVER leaves device

↓

Flutter app creates CSR

↓

NestJS signs CSR using Intermediate CA

↓

Device receives client certificate

↓

mTLS works

Step 1 – Generate Keypair in Android Keystore

Flutter:

```
static const platform =  
  MethodChannel('com.example.security/keystore');  
  
Future<void> generateKeyPair() async {  
  await platform.invokeMethod('generateKeyPair');  
}
```

Android (Kotlin):

```
val keyPairGenerator = KeyPairGenerator.getInstance(  
  KeyProperties.KEY_ALGORITHM_RSA,  
  "AndroidKeyStore"  
)
```

Result:

- Private key is hardware protected
- Private key is non-exportable
- APK extraction becomes useless

Step 2 – Generate CSR

Gradle:

```
implementation "org.bouncycastle:bcprov-jdk18on:1.78"  
implementation "org.bouncycastle:bcpkix-jdk18on:1.78"
```

Step 3 – Send CSR from Flutter to NestJS

```
final response = await http.post(  
  Uri.parse('https://api.yourdomain.com/device/register'),  
  body: {
```

```
    'csr': csrPem,  
  },  
);
```

Step 4 – Sign CSR in NestJS

```
openssl x509 -req -in device.csr -CA intermediate.crt -CAkey intermediate.key
```

Recommended Production Architecture

Flutter App



Android Keystore generates keypair



CSR sent to NestJS



Intermediate CA signs cert



Client cert returned



mTLS



JWT issued

Security Advantages

- Hardware-backed keys
- Non-exportable private keys
- Device-unique certificates
- mTLS authentication
- Certificate pinning
- JWT after mTLS

Final Insight

This architecture transforms your app into a cryptographically authenticated device identity system.